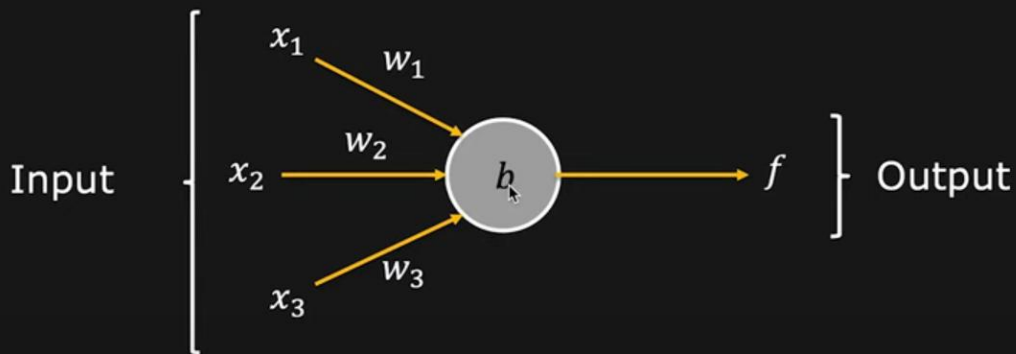


POCKET PERCEPTRON

Τι κάνει το perceptron;

Το perceptron δέχεται διάφορες εισόδους $x_1, x_2, \dots, x_n$ και δημιουργεί ένα δυαδικό αποτέλεσμα ή μια απόφαση. Για να το πετύχει αυτό πολλαπλασιάζει τις εισόδους με τα βάρη $w_1, w_2, \dots, w_n$ και συγκρίνει το αποτέλεσμα το Bias b (threshold). Με βάση την σύγκριση που γίνεται βγαίνει το αποτέλεσμα που είναι είτε 0 είτε 1.

Takes several inputs and gives a single binary output (Decision)



$$f = \begin{cases} 0 & \text{if } \sum_j w_j x_j \leq -b \\ 1 & \text{if } \sum_j w_j x_j > -b \end{cases}$$

where, $w_j = \text{weights}$
 $b = \text{bias (threshold)}$

Γράφουμε όλα τα βάρη με vector w , όλες τις εισόδους με vector x . Το perceptron εφαρμόζει μια

$$\text{Let: } z = w \cdot x + b$$

συνάρτηση για το z . Στην περίπτωση του perceptron το αποτέλεσμα είναι απλό.

$$\text{Activation: } a = f(z) = \begin{cases} 0 & \text{if } z \leq 0 \\ 1 & \text{if } z > 0 \end{cases}$$

Η συνάρτηση ενεργοποίησης του perceptron είναι μια συνάρτηση βαθμίδας (step function).

```

function [] = pocket_perceptron(input)

% Δεδομένα εισόδου
x1 = input(:,1); % Χαρακτηριστικό x1
x2 = input(:,2); % Χαρακτηριστικό x2
target = input(:,3); % Στόχοι (-1 ή 1)

% Αρχικοποίηση
num_samples = length(input); % Αριθμός δειγμάτων
learning_rate = 0.01; % Ρυθμός εκμάθησης
max_epochs = 1000; % Μέγιστος αριθμός επαναλήψεων (epochs)
bias = 1; % Bias
weights = [0; 0; 0]; % Αρχικοποίηση βαρών σε 0
best_weights = weights; % Καλύτερα βάρη (Pocket Weights)
max_correct = 0; % Μέγιστος αριθμός σωστών προβλέψεων

% Δημιουργία figure για σχεδίαση
figure('Name', 'Pocket Perceptron');
hold on;
grid on;
class_minus = scatter(x1(target == -1), x2(target == -1), 120, 'r', 'x',
'LineWidth', 2); % Κατηγορία -1
class_plus = scatter(x1(target == 1), x2(target == 1), 120, 'b', 'o',
'LineWidth', 2); % Κατηγορία 1 Για την εκτύπωση μηνυμάτων στη γραμμή εντολών.
axis([min(x1)-1, max(x1)+1, min(x2)-1, max(x2)+1]); % Ρυθμίσεις άξονα

% Εκπαίδευση Pocket Perceptron
for epoch = 1:max_epochs
    correct_predictions = 0; % Καταμέτρηση σωστών προβλέψεων Για την εκτύπωση
    μηνυμάτων στη γραμμή εντολών.

    for i = 1:num_samples
        % Υπολογισμός πρόβλεψης
        weighted_sum = (bias * weights(3)) + (x1(i) * weights(1)) + (x2(i) *
weights(2));
        prediction = sign(weighted_sum); % Υπολογισμός πρόβλεψης

        % Αντιμετώπιση του 0
        if prediction == 0
            prediction = -1;
        end

        % Υπολογισμός σφάλματος

```

```

        error = target(i) - prediction;

        % Αν πρόβλεψη σωστή
        if error == 0
            correct_predictions = correct_predictions + 1;
            if correct_predictions > max_correct
                max_correct = correct_predictions;
                best_weights = weights; % Αποθήκευση καλύτερων βαρών
            end
        else
            % Ενημέρωση βαρών αν λάθος πρόβλεψη
            weights(1) = weights(1) + learning_rate * x1(i) * error;
            weights(2) = weights(2) + learning_rate * x2(i) * error;
            weights(3) = weights(3) + learning_rate * bias * error;
        end
    end

    % Σχεδίαση διαχωριστικής γραμμής
    if weights(2) ~= 0
        line_x = [min(x1), max(x1)];
        line_y = -((weights(1) * line_x + bias * weights(3)) / weights(2));
        plot(line_x, line_y, 'k--'); % Μαύρη διακεκομμένη γραμμή για τρέχουσα
        γραμμή
    else
        x_vertical = -weights(3) / weights(1);
        plot([x_vertical, x_vertical], [min(x2)-1, max(x2)+1], 'k--'); %
        Κατακόρυφη γραμμή
    end
    drawnow;

    % Τερματισμός αν όλες οι προβλέψεις είναι σωστές
    if correct_predictions == num_samples
        fprintf('Όλες οι προβλέψεις είναι σωστές στο epoch %d.\n', epoch);
        break;
    end
end

% Σχεδίαση τελικής διαχωριστικής γραμμής
if best_weights(2) ~= 0
    final_line_y = -((best_weights(1) * line_x + bias * best_weights(3)) /
    best_weights(2));
    final_line = plot(line_x, final_line_y, 'r-', 'LineWidth', 2); % Κόκκινη
    γραμμή για τελική διαχωριστική γραμμή
else
    x_vertical_final = -best_weights(3) / best_weights(1);

```

```
        final_line = plot([x_vertical_final, x_vertical_final], [min(x2)-1,
max(x2)+1], 'r-', 'LineWidth', 2); % Κατακόρυφη τελική γραμμή
    end

    % Δημιουργία legend
    legend([class_minus, class_plus, final_line], ...
        {'Class -1', 'Class 1', 'Final Decision Boundary'}, ...
        'Location', 'best');

    % Εμφάνιση αποτελεσμάτων
    fprintf('Καλύτερα βάρη (Pocket Weights):\n');
    fprintf('w1: %.4f, w2: %.4f, Bias: %.4f\n', best_weights(1), best_weights(2),
best_weights(3));
    fprintf('Μέγιστος αριθμός σωστών προβλέψεων: %d/%d\n', max_correct, num_samples);
end
```

Ανάλυση

```
% Δεδομένα εισόδου
x1 = input(:,1); % Χαρακτηριστικό x1
x2 = input(:,2); % Χαρακτηριστικό x2
target = input(:,3); % Στόχοι (-1 ή 1)
```

```
% Αρχικοποίηση
num_samples = length(input); % Αριθμός δειγμάτων
learning_rate = 0.01; % Ρυθμός εκμάθησης
max_epochs = 1000; % Μέγιστος αριθμός επαναλήψεων (epochs)
bias = 1; % Bias
weights = [0; 0; 0]; % Αρχικοποίηση βαρών σε 0
best_weights = weights; % Καλύτερα βάρη (Pocket Weights)
max_correct = 0; % Μέγιστος αριθμός σωστών προβλέψεων
```

```
% Δημιουργία figure για σχεδίαση
figure('Name', 'Pocket Perceptron');
hold on;
grid on;
class_minus = scatter(x1(target == -1), x2(target == -1), 120, 'r', 'x', 'LineWidth', 2); %
Κατηγορία -1
class_plus = scatter(x1(target == 1), x2(target == 1), 120, 'b', 'o', 'LineWidth', 2); % Κατηγορία
1
axis([min(x1)-1, max(x1)+1, min(x2)-1, max(x2)+1]); % Ρυθμίσεις άξονα
```

Το βασικό κομμάτι του αλγορίθμου.

```
% Εκπαίδευση Pocket Perceptron
for epoch = 1:max_epochs
    correct_predictions = 0; % Καταμέτρηση σωστών προβλέψεων

    for i = 1:num_samples
        % Υπολογισμός πρόβλεψης
        weighted_sum = (bias * weights(3)) + (x1(i) * weights(1)) + (x2(i) * weights(2));
        prediction = sign(weighted_sum); % Υπολογισμός πρόβλεψης

        % Αντιμετώπιση του 0
        if prediction == 0
            prediction = -1;
        end

        % Υπολογισμός σφάλματος
        error = target(i) - prediction;

        % Αν πρόβλεψη σωστή
        if error == 0
            correct_predictions = correct_predictions + 1;
            if correct_predictions > max_correct
                max_correct = correct_predictions;
                best_weights = weights; % Αποθήκευση καλύτερων βαρών
            end
        else
            % Ενημέρωση βαρών αν λάθος πρόβλεψη
            weights(1) = weights(1) + learning_rate * x1(i) * error;
            weights(2) = weights(2) + learning_rate * x2(i) * error;
            weights(3) = weights(3) + learning_rate * bias * error;
        end
    end
end
```

Ο εξωτερικός βρόχος τρέχει για μέγιστο αριθμό επαναλήψεων. Κάθε φορά γίνεται μηδενισμός των σωστών προβλέψεων. Υπολογισμός του `weighted_sum`, το άθροισμα του γινομένου των εισόδων και από τα βάρη με την προσθήκη του γινομένου `bias` και του βάρους. Η πρόβλεψη θα είναι είτε 1 αν `weighted_sum > 0` αλλιώς -1. Γίνεται υπολογισμός του `error` και στην συνέχεια γίνεται ενημέρωση βαρών. Με τον κανόνα

$$w_j = w_j + \eta \cdot x_j \cdot \text{error}$$

όπου η ο ρυθμός εκμάθησης που έχουμε ορίσει.

```

% Σχεδίαση διαχωριστικής γραμμής
if weights(2) ~= 0
    line_x = [min(x1), max(x1)];
    line_y = -((weights(1) * line_x + bias * weights(3)) / weights(2));
    plot(line_x, line_y, 'k--'); % Μαύρη διακεκομμένη γραμμή για τρέχουσα γραμμή
else
    x_vertical = -weights(3) / weights(1);
    plot([x_vertical, x_vertical], [min(x2)-1, max(x2)+1], 'k--'); % Κατακόρυφη γραμμή
end
drawnow;

```

```

% Σχεδίαση τελικής διαχωριστικής γραμμής
if best_weights(2) ~= 0
    final_line_y = -((best_weights(1) * line_x + bias * best_weights(3)) / best_weights(2));
    final_line = plot(line_x, final_line_y, 'r-', 'LineWidth', 2); % Κόκκινη γραμμή για τελική
διαχωριστική γραμμή
else
    x_vertical_final = -best_weights(3) / best_weights(1);
    final_line = plot([x_vertical_final, x_vertical_final], [min(x2)-1, max(x2)+1], 'r-',
'LineWidth', 2); % Κατακόρυφη τελική γραμμή
end

```

```

% Τερματισμός αν όλες οι προβλέψεις είναι σωστές
if correct_predictions == num_samples
    fprintf('Όλες οι προβλέψεις είναι σωστές στο epoch %d.\n', epoch);
    break;
end
end

```

```
% Δημιουργία legend
legend([class_minus, class_plus, final_line], ...
       {'Class -1', 'Class 1', 'Final Decision Boundary'}, ...
       'Location', 'best');

% Εμφάνιση αποτελεσμάτων
fprintf('Καλύτερα βάρη (Pocket Weights):\n');
fprintf('w1: %.4f, w2: %.4f, Bias: %.4f\n', best_weights(1), best_weights(2), best_weights(3));
fprintf('Μέγιστος αριθμός σωστών προβλέψεων: %d/%d\n', max_correct, num_samples);

end
```

Για την οπτικοποίηση του κώδικα χρησιμοποίησα

<https://carbon.now.sh/>

Μεταβλητές

x1	Χαρακτηριστικό 1 (πρώτη στήλη του πίνακα εισόδου).
x2	Χαρακτηριστικό 2 (δεύτερη στήλη του πίνακα εισόδου).
target	Ετικέτες στόχου (-1 ή 1) για την ταξινόμηση, τρίτη στήλη του πίνακα εισόδου.
num_samples	Συνολικός αριθμός δειγμάτων στα δεδομένα εισόδου.
learning_rate	Ρυθμός εκμάθησης για την προσαρμογή των βαρών κατά την εκπαίδευση.
max_epochs	Μέγιστος αριθμός επαναλήψεων για την εκπαίδευση.
bias	σταθερή τιμή που χρησιμοποιείται στην εξίσωση της διαχωριστικής γραμμής
weights	Τρέχοντα βάρη του μοντέλου
best_weights	Καλύτερα βάρη που βρέθηκαν κατά την εκπαίδευση (μηχανισμός rocket).
max_correct	Μέγιστος αριθμός σωστών προβλέψεων που έχουν καταγραφεί μέχρι τώρα.
correct_predictions	Αριθμός σωστών προβλέψεων στο τρέχον epoch.
weighted_sum	άθροισμα για ένα δείγμα (γραμμικός συνδυασμός βαρών και χαρακτηριστικών).
prediction	Πρόβλεψη για ένα δείγμα βάσει του αθροίσματος (συνάρτηση sign).
error	Σφάλμα που υπολογίζεται ως target - prediction για ένα δείγμα.
line_x	Εύρος τιμών x που χρησιμοποιούνται για τη σχεδίαση της διαχωριστικής γραμμής.
line_y	Αντίστοιχες τιμές y για τη γραμμή διαχωρισμού.
x_vertical	x-συντεταγμένη για μια κατακόρυφη διαχωριστική γραμμή (αν ισχύει).
final_line_y	Τελικές τιμές y για τη γραμμή διαχωρισμού με τα καλύτερα βάρη.
final_line	Τελική διαχωριστική γραμμή.
x_vertical_final	Τελική x-συντεταγμένη για μια κατακόρυφη διαχωριστική γραμμή με τα καλύτερα βάρη.

Συναρτησεις matlab

length	Για τον υπολογισμό του αριθμού δειγμάτων.
scatter	Για τη σχεδίαση δεδομένων σε διάγραμμα διασποράς.
axis	Για τη ρύθμιση των ορίων των αξόνων.
sign	Για τον υπολογισμό του πρόσημου μιας τιμής (πρόβλεψη).
plot	Για τη σχεδίαση γραμμών (διαχωριστική γραμμή).
hold on	Για τη διατήρηση πολλαπλών γραφικών στο ίδιο διάγραμμα.
legend	Για την προσθήκη επεξηγήσεων (υπομνημάτων) στο διάγραμμα.
fprintf	Για την εκτύπωση μηνυμάτων στη γραμμή εντολών.
figure	Χρησιμοποιείται για να δημιουργήσει ένα νέο παράθυρο γραφήματος
hold on	Επιτρέπει τη διατήρηση πολλαπλών γραφικών στο ίδιο διάγραμμα, χωρίς να διαγράφεται το προηγούμενο περιεχόμενο.
grid on	Ενεργοποιεί το πλέγμα στο διάγραμμα, καθιστώντας το πιο ευανάγνωστο.
drawnow	Ενημερώνει το διάγραμμα σε πραγματικό χρόνο, έτσι ώστε να εμφανίζονται οι αλλαγές άμεσα.

Αναφορες

https://www.ling.upenn.edu/courses/Fall_2007/cogs501/Gallant1990.pdf

<https://www.codecademy.com/learn/perceptrons-and-neural-nets-skill-path/modules/perceptrons-skill-path/cheatsheet>

https://www.youtube.com/watch?v=OFbnpY_k7js&t=92s

<https://www.youtube.com/watch?v=aiDv1NPdXvU>

<https://www.youtube.com/watch?v=OVHc-7GYRo4>

<https://stats.stackexchange.com/questions/51200/pocket-algorithm-for-training-perceptrons>

<https://positronicsengineer.wordpress.com/home/pocket-learning/>

<https://www.youtube.com/watch?v=4Gac5l64LM4>

<https://www.youtube.com/watch?v=i1G7PXZMnSc>

<https://simran-tinani.medium.com/the-perceptron-learning-algorithm-is-one-of-the-simplest-machine-learning-algorithms-and-a-6ab04c5772ad>

<https://www.codeproject.com/Articles/1229772/Machine-Learning-Basics-Pocket-Learning-Algorithm>

<https://archive.ics.uci.edu/datasets>

<https://stackoverflow.com/questions/4984532/unicode-characters-in-matlab-source-files>

<https://www.mathworks.com/help/matlab/ref/fprintf.html>

<https://www.mathworks.com/help/matlab/ref/hold.html>

<https://www.mathworks.com/help/matlab/ref/double.sign.html>

https://www.mathworks.com/help/matlab/ref/double.length.html?s_tid=doc_ta

https://www.mathworks.com/help/matlab/ref/scatter.html?s_tid=doc_ta

https://www.mathworks.com/help/matlab/ref/axis.html?s_tid=doc_ta

https://www.mathworks.com/help/matlab/ref/plot.html?searchHighlight=plot&s_tid=srchtitle_support_results_1_plot#d126e1302169

<https://www.mathworks.com/help/matlab/ref/drawnnow.html>

<https://www.mathworks.com/help/matlab/ref/grid.html>

<https://www.mathworks.com/help/matlab/ref/figure.html>

<https://www.mathworks.com/help/matlab/ref/legend.html>